Algorithm 1 Pseudocode for NextLat's loss function in PyTorch syntax. The loss functions `cross_entropy_loss()`, `smooth_l1_loss()`, and `KL_loss()` are implemented externally.

```
import torch.nn as nn
from copy import deepcopy

def loss(batch, targets):
    # batch:(B,T), targets: (B,T)
    # embedding dimension = D, vocabulary size = V
    batch_size, seq_len = batch.shape

    # Encode sequences into latent states
    hidden_states = Transformer(batch) # (B,T,D)

    # Compute next-token loss
    logits_post = Output_Head(hidden_states) # (B,T,V)
    loss_next = cross_entropy_loss(logits_post, targets)

    next_tokens = batch
    next_states = hidden_states
    current_states = hidden_states
    loss_next_h = 0
    loss_kl = 0

    # Recursive multi-step predictions, default multi_step_horizon=1;
    # only 1-step prediction
    for _ in range(multi_step_horizon):
        # Shift hidden states back by 1
        current_states = current_states[:, :-1]
        # Shift next tokens and hidden state targets forward by 1
        next_tokens = next_tokens[:, 1:]
        next_states = next_states[:, 1:]
        logits_post = logits_post[:, 1:]
        # Predict next hidden state using latent dynamics model
        # i.e., (h_t, x_{t+1}) -> \hat{h}_{t+1}
        pred_next_states = Latent_Dynamics(current_states, next_tokens) #(B,T,D)
        # Compute next-hidden loss using detached next states as targets
        # i.e., ||h_{t+1} - \hat{h}_{t+1}||^2
        loss_next_h += smooth_l1_loss(pred_next_states, next_states.detach())

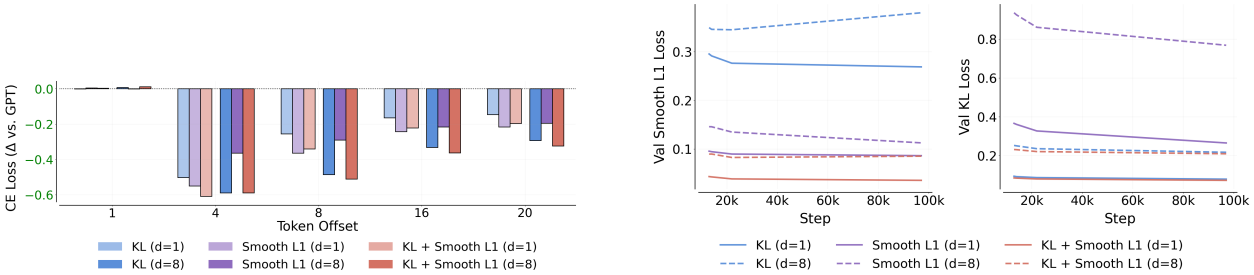
        # Compute KL loss using detached output head
        # i.e., KL[p(h_{t+1}) || p(\hat{h}_{t+1})]
        logits_prior = deepcopy(Output_Head)(pred_next_states) # (B,T,V)
        loss_kl += KL_loss(logits_post.detach(), logits_prior) # detach posterior

        current_states = pred_next_states

    loss_next_h = loss_next_h / multi_step_horizon
    loss_kl = loss_kl / multi_step_horizon
    # overall NextLat loss
    return loss_next + next_h_lambda * loss_next_h + kl_lambda * loss_kl
```

D Ablation Studies

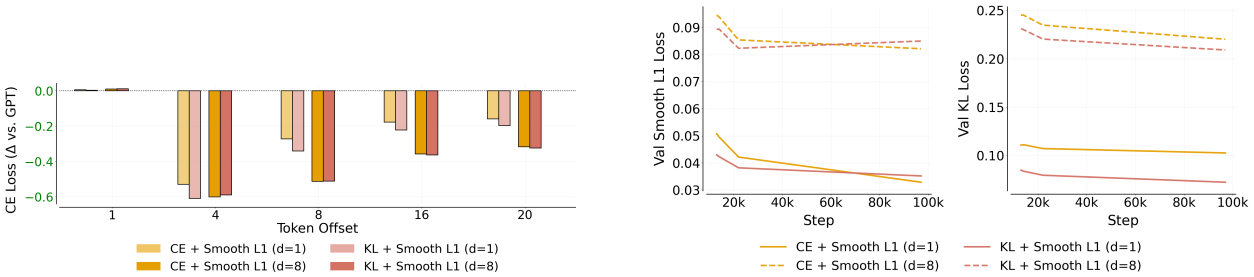
In this section, we ablate the key design choices of NextLat. Specifically, we study the effects of the KL ($\mathcal{L}_{\text{KL}}$) and Smooth L1 ($\mathcal{L}_{\text{next-h}}$) losses, as well as the use of stop-gradients on the target ($\mathbf{sg}[\mathbf{h}_{t+i}]$) in the Smooth L1 loss (Equation 3) and on the output head ($p_\theta^{\mathbf{sg}}(\cdot)$) in the KL loss (Equation 4). We focus our investigations primarily on TinyStories, analyzing how these design choices affect the predictive quality of learned hidden states under linear probing and validation loss behavior.



(a) (b)

Figure 12: **KL and Smooth L1 Loss Ablations:** (a) TinyStories probe performance under ablations of the KL and Smooth L1 losses. (b) TinyStories validation Smooth L1 and KL losses.

We first isolate the effects of the KL and Smooth L1 losses in NextLat. Note that $\mathcal{L}_{\text{next-h}}$ is required for belief-state convergence, while $\mathcal{L}_{\text{KL}}$ serves as a complementary supervision signal. In Figure 12a, we observe that at $d = 1$, using Smooth L1 loss alone achieves the strongest probe performance 20 tokens ahead relative to GPT. At $d = 8$, however, the combined KL + Smooth L1 objective (i.e., the full NextLat design) performs best, suggesting that the Smooth L1 loss is critical for learning long-range predictive representations, while the KL loss becomes increasingly beneficial at larger horizons. Furthermore, Figure 12b shows that the combined KL + Smooth L1 objective achieves lower validation KL and Smooth L1 losses than optimizing either objective alone. This confirms that the two losses are complementary and provide mutually beneficial supervision.

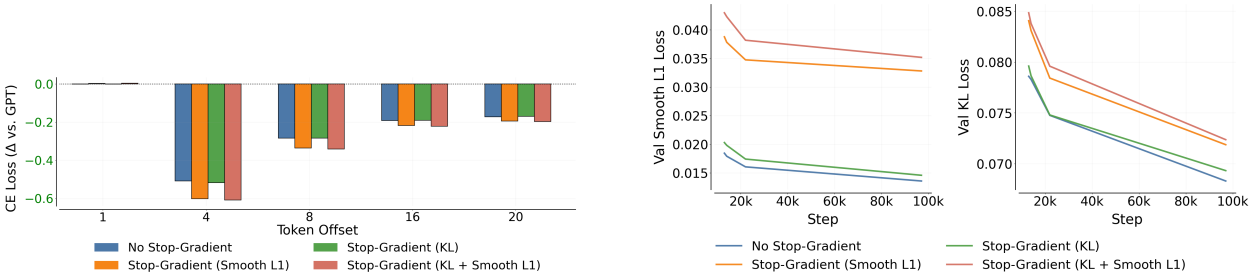


(a) (b)

Figure 13: **KL and CE Loss Ablations:** (a) TinyStories probe performance when replacing the KL loss in NextLat with a CE loss. (b) TinyStories validation Smooth L1 and KL losses.

Next, we compare the KL loss against cross-entropy (CE) as the complementary token-space supervision signal. Note that the CE loss corresponds exactly to the multi-token prediction objective used in JTP and MTP. Unlike the KL loss, which performs self-distillation by matching the predicted token distribution to the model's own softmax distribution under the target hidden state (see Equation 4), the CE loss directly supervises the predicted latent state using the ground-truth next token. As shown in Figure 13a, the original NextLat objective (KL + Smooth L1) achieves substantially better probe performance than CE + Smooth L1 at $d = 1$, though the gap becomes marginal at $d = 8$. Figure 13b shows that across both $d = 1$ and $d = 8$, KL + Smooth L1 achieves lower validation KL loss, but slightly

worse Smooth L1 loss, than CE + Smooth L1. Overall, these results suggest that at shallow multi-step prediction horizons, the dense distribution-level supervision provided by KL matching induces more predictive representations than CE. However, this advantage appears to diminish at larger horizons. It is also important to note that the KL loss incurs higher memory overhead than CE loss. This is because KL requires materializing full logits and softmax distributions over the vocabulary, whereas modern fused CE implementations only need to materialize probabilities at the target token index. More systematic studies are needed, especially at larger model scales, to determine whether KL or CE is preferable as a complementary supervision signal. For the scope of this work, we stick to using KL loss in all experiments.



(a) (b)

Figure 14: **Stop-Gradient Ablations:** (a) TinyStories probe performance under different stop-gradient configurations. (b) TinyStories validation Smooth L1 and KL losses. Note that all models here use both KL and Smooth L1 losses and differ only in where stop-gradients are applied in those loss components. "Stop-Gradient (Smooth L1)" applies stop-gradient only to the target hidden state in the Smooth L1 loss, "Stop-Gradient (KL)" applies stop-gradient only to the output head in the KL loss, and "Stop-Gradient (KL + Smooth L1)" applies stop-gradients to both components. All models use $d = 1$ here.

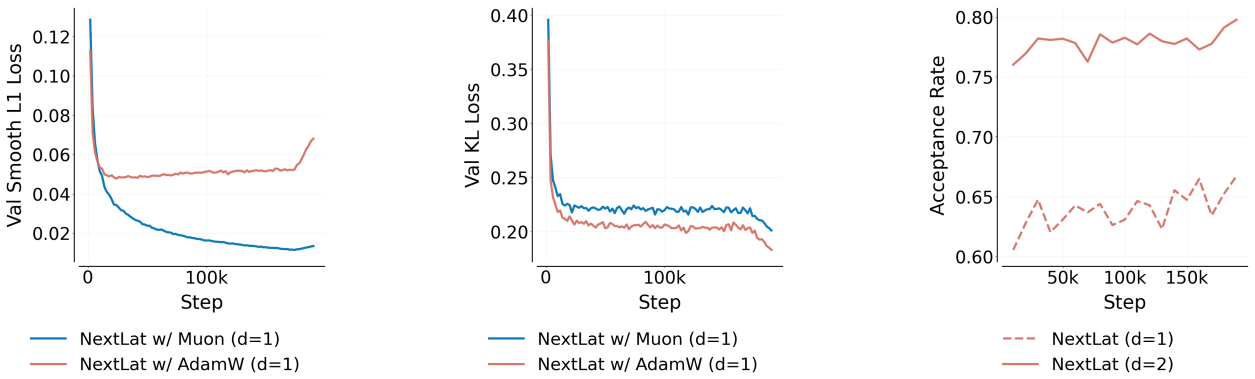
Next, we study the use of stop-gradients in Figure 14 with $d = 1$. While Figure 14b shows that removing the stop-gradient on the Smooth L1 loss improves validation Smooth L1 and KL losses slightly (~ 0.02), it does not improve the predictive quality of the learned representations. As shown in Figure 14a, applying stop-gradients to both the target in the Smooth L1 loss and the output head ($p_\theta^{\mathbf{sg}}(\cdot)$) in the KL loss yields the best probing performance across all token offsets. Stop-gradients also improve efficiency by reducing the number of backward passes through the transformer. In particular, the KL loss introduces an additional forward and backward pass through the output head, which can be computationally expensive in language models due to large vocabulary sizes[8]. Removing this extra backward pass significantly improves both speed and memory efficiency. Empirically, on tasks like Countdown and Path-Star graph, we also observe that the stop-gradient (especially on the Smooth L1 loss) is essentially for high accuracy.

Overall, these ablations suggest that NextLat's effectiveness arises from the combination of Smooth L1 and KL losses together with carefully placed stop-gradients, all of which contribute to learning predictive belief-state representations in transformers.

E NextLat Pretraining Quirks

In this section, we expose some optimization quirks that we faced during pretraining with NextLat. First, Figure 15a shows vastly different Smooth L1 loss trajectories when using Muon [Jordan et al., 2024] versus AdamW [Loshchilov and Hutter, 2019] optimizer under the same learning rate schedule (using the Muon update scaling rule proposed by Liu et al. [2025]). One possible explanation is that Muon's orthogonalized update rule enables more efficient latent representation shaping and next-latent prediction optimization. However, Figure 15b shows that AdamW achieves better KL loss optimization than Muon.

[8]Note that multi-token prediction models also incur an additional forward+backward pass through the output head for each extra token prediction.



(a) (b) (c)

Figure 15: **FineWeb-Edu Pretraining Curves:** (a) Validation smooth L1 loss during training. (b) Validation KL loss during training. (c) Speculative decoding acceptance rate over a 5-token lookahead horizon (excluding the next-token prediction).

Importantly, both optimizers exhibit **an increase in Smooth L1 loss near the end of training** corresponding to the learning rate cooldown stage, though the effect is substantially more pronounced for AdamW. The cause of this behavior remains unclear. We explored several potential fixes to no avail, including:

1. replacing Smooth L1 with MSE loss,
2. removing stop-gradients from $\mathcal{L}_{\text{next-h}}$,
3. tuning loss coefficients $\lambda_{\text{next-h}}$ and λ_{KL},
4. disabling learning rate decay for the latent transition model, and
5. disabling weight decay for the latent transition model.

We leave a deeper investigation of this phenomenon to future work. To our reassurance, the increase in smooth L1 loss does not appear to degrade the coherence of the latent transition model. As shown in Figure 15c, the speculative decoding acceptance rate over a lookahead horizon of 5 tokens continues to improve throughout training, even during the phase where the smooth L1 loss increases under AdamW. This suggests that the rise in smooth L1 loss may reflect changes in the scale or geometry of the latent states, rather than degradation in the token-level coherence of the latent transition dynamics.

On a side note, we observed gradient norm spikes that led to training instability when training MTP with Muon, which unfortunately limited our pretraining analysis to AdamW (despite Muon showing a more favorable learning trajectory with NextLat).

F Extra Experiment Details

In this section, we provide additional experimental setup details and supplementary results for the experiments in the main body. All experiments were run on NVIDIA RTX A5000, NVIDIA H100 NVL, and NVIDIA B200 GPUs. Table 5 summarizes the training, model, and NextLat hyperparameters across all experimental domains.

The hyperparameters reported in Table 5 were chosen through a small-scale search, exploring $\lambda_{\text{next-h}} \in \{1.0, 2.0\}$ and $\lambda_{\text{KL}} \in \{0.1, 1.0\}$, guided by empirical observations and intuition. Encouragingly, we find that NextLat performs robustly over a wide range of settings. In particular, λ_{KL} requires minimal tuning; $\lambda_{\text{KL}} = 1.0$ works decently across all tasks. After all, it primarily serves as a complementary alignment objective. Likewise, $\lambda_{\text{next-h}} = 1.0$ is effective in most cases, though slightly higher values (e.g., $\lambda_{\text{next-h}} = 2.0$) was beneficial when the next-latent regression loss is of much smaller scale than the token-level losses (i.e., $\mathcal{L}_{\text{next-token}}$ and $\mathcal{L}_{\text{KL}}$).

For MTP and JTP, we sweep multi-token prediction loss weights $\lambda_{\text{MTP}} \in \{0.1, 0.2, 0.4, 0.6, 0.8, 1.0\}$ to ensure fair